

Report on model-based software design assignment

Salvatore Junior Curello
Matr. 268066



Politecnico di Torino
A.Y. 2020 – 2021

Model-based Software Design Assignment Report

Salvatore Junior Curello
s268066

1. Simulink project work-logic

The aim of this assignment is to implement a level indicator, which is composed of an Ultrasonic Ranging Sensor that is used to measure the distance of a target from our measurement system and based on the acquired information, we have a Ranging Estimator that must produce a level information.

Due to the nature of the system that we are solving we used a **FixedStepDiscrete** Time solver since the Ultrasonic Ranging Sensor is providing as output a square waveform, so it is not recommended to adopt a continuous time solver. For this reason, we set the entire model to operate as FixedStepDiscrete. Moreover, we must set the fundamental step size to a value that allow us to represent our time accurately. We adopted 1 microsecond as fundamental sample time in order to have more measurements accuracy.

The model is structured as follow:

- *Controller*: this is where we process the echo signal, and we generate the level output as well as the trigger output.
- *Plant*: It is used to simulate the Ultrasonic Ranging Sensor, it is feed with an input coming from the controller that is the trigger signal and another input that is the distance controlled by a knob. The output that it will produce is the echo signal.

All the inputs and the outputs have been configured as *uint8* so that the resulting in code generation will not occupy too much memory, the only input that is using a different DataType is the *distance* which has been configured as double.

The model of the Ultrasonic Ranging Sensor is composed of a Stateflow in which, once a distance is received, a saturation process will be implemented in the sense that for each measure lower than 2cm and higher than 400cm the value 2 or 400 centimeters will be assigned respectively.

When the *trigger_input* signal goes to 1 we go from the **OFF** state to the **ON** state in which we count, through a counter, the number of μs that the signal is equal to 1. Once the *trigger_input* signal goes again to 0 we check if the value of the counter is equal or greater than 10 to verify if the pulse duration is at least 10 μs .

If this condition is verified, we go into the **ON1** state in which we set the *echo_output* signal to 1, for a duration equal to *distance* multiplied by 58, which will be the output of the Ultrasonic Ranging Sensor.

The controller has been implemented by a Stateflow which is composed of three parallel states: **TRIGGER**, **LED** and **LOGIC_OPERATOR**.

The following table shows the variables that we can find in these states together with their description.

Name	Scope	DataType	Description
<i>echo_input</i>	Input	uint8	It is equal to the value of the input at port 1. It is used to understand when the echo signal is on or off.
<i>level</i>	Output	uint8	Used to send the value of the level output to the port 1.
<i>trigger_output</i>	Output	uint8	Used to send the value of the trigger output to the port 2.
<i>counter</i>	Local	uint16	Used to count how many microseconds <i>echo_input</i> is equal to 1.
<i>dist_v</i>	Local	double	Array used to save four pulses duration.
<i>mean</i>	Local	double	It represents the average of the four pulses duration saved in <i>dist_v</i> divided by 58 to get the relative distance in centimeters.
<i>i</i>	Local	uint8	Index of <i>dist_v</i> used to acquire only four measures.
<i>up</i>	Local	double	It indicates the seconds that the level output must be equal to 1.
<i>down</i>	Local	double	It indicates the seconds that the level output must be equal to 0.

LOGIC OPERATOR

This state starts from the **INITIAL_STATE** in which we initialize *dist_v*, *i*, *counter* and *mean*. If the value of *echo_input* will be equal to zero for more than 0.06001 seconds we go in the **ERROR** state since no signal is arriving, otherwise once the value of *echo_input* goes to 1 we reach another state called **COUNTER**. In this state we count the number of microseconds that *echo_input* was equal to 1 and we save it in *counter*. Then once *echo_input* goes to 0 we pass in the **SAVE_MEASURE** state where we save the value of *counter* in the array *dist_v* at position *i*.

To leave this state we must check if one of the following transactions, written in order of priority, is verified:

1. *i*==4: if *i* is equal to 4 it means that we already have four measures inside the array so we can proceed to the next step.
2. *echo_input*==1: we do not have four measures yet. A new pulse is coming so we go back in the previous state **COUNTER** in order to compute the time duration of the new pulse but before to do that we initialize the variable counter to zero and we increment the value of the index *i*.
3. *after*(0.06001, sec): if none of the previous conditions is verified it means that we are not receiving any value from the *echo_input* signal so we move to the **ERROR** state.

The next state is the **MEAN** state in which we sum the values of the four measures saved in the *dist_v* and we divide this value for 232 (4*58) in order to compute the average of the measures and convert them in a distance measured in centimeters. We save this value in the variable *mean*. Furthermore, since we are implementing a moving average we cannot lose the information of the last 3 measures arrived, for this reason, we implement a sliding window moving behaviour.

We set the value of *i* to 4 so that we will save the next measure always in the fourth position of the array.

Now we check the value of *mean*, if it is greater or equal to 10 and less or equal to 100, we move in another state called **IN_RANGE**, instead if this condition is not verified we reach the **OUT_RANGE** state.

Once we enter in the **IN_RANGE** state we multiply the value of mean for 0.001 and we save it in the *up* variable. Then we compute the value of *down* by subtracting the *up* value to 0.1.

Instead, if the measure is out range we set both *up* and *down* values to 0.25.

When we are inside one of these states (**IN_RANGE** or **OUT_RANGE**) we wait that the value of the *echo_input* will become 1.

Once this condition is verified, we move again in the **COUNTER** state (caring of to set counter=0 as an action of the transaction) in order to count the new arriving pulse duration. Otherwise, if after 0.06001 sec we are still inside in the state **IN_RANGE** or **OUT_RANGE**, it means that a new measure is not coming so we move in the **ERROR** state. In this state we set both *up* and *down* values equal to 0.5.

TRIGGER

This state is composed of two substates in which we generate the pulse of the trigger. In particular, the trigger signal stays on the **ON** state for 10 μ s and it stays on the **OFF** state for 60ms, to avoid echoes. The period of the trigger signal is 0.06001 second.

LED

In this state the level output switch from 0 (**OFF** state) to 1 (**ON** state) and from 1 to 0 for an amount of seconds which is equal to *down* and *up* respectively. In particular, before moving from the **ON** state to the **OFF** state we check if the value of *down* is different from zero because if we give a distance value of 100cm the led should always stay high.

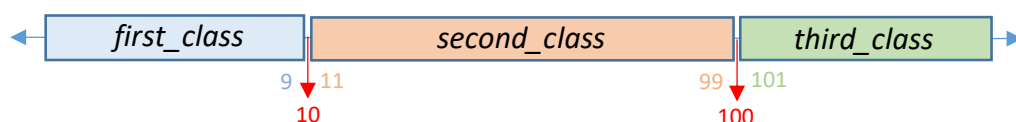
Futhermore, in order to avoid any error, we assigned as initial value of *up* and *down* a value of 2 seconds, in this way we can receive the first measure before that we enter in the **OFF** state.

2. Google test

To perform a Google Test, the procedure that we used is to write a test using simulink test and execute it by collecting input stimuli, expected output and trigger output in such a way that they can be reused after code generation take place and the generated code is tested using Google test. For this purpose, we created a *TestHarness* on our unit under test that is the controller mentioned above. Inside the TestHarness we created an input stimuli using Stateflow.

To improve the quality of the test, we partitioned the input domain to find equivalence classes, we identify their boundaries, and we generate an input to test both equivalence classes and their boundaries.

Based on our requirements specifications, we identified three different equivalence classes: *first_class* ($x < 10$), *second_class* ($x \geq 10$ and $x \leq 100$), *third_class* ($x > 100$), and taking into account their boundaries, we generated an input which values are: 9, 10, 11, 99, 100, 101 and a suitable input for the error case.



The output of the Stateflow feeds the *echo_input* port of the controller.

In order to collect the input stimuli, the expected output and the trigger output, we used three “*To Workspace*” blocks which allow us to sample the value of a signal and create an object that contains the time sequence of values assumed by a particular signal during the simulation.

The code has been generated using algorithm export process based on compact file placement, reusable interface, and ARM/cortex M device.

The coverage metric that we used is the *decision* coverage so we are going to access during Simulink simulation the quality of the test in terms of how many branches are covered out of the possible decisions available in our Simulink model.

With a simulation time of 2.7s we already reach 100% of coverage in Simulink test. The figure below shows the output of the controller which is consistent with what we were expecting.

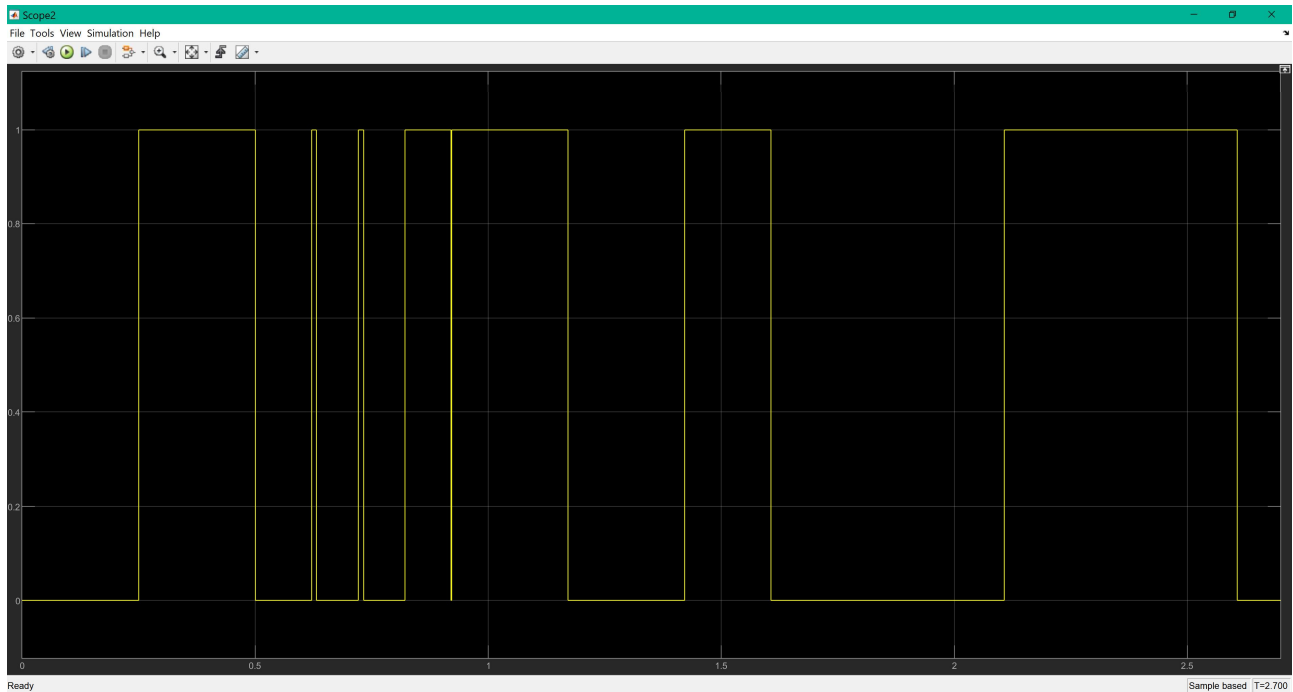


Figure 1. We tested respectively: 9(out_range), 10(in_range), 11(in_range), 99(in_range), 100(in_range), 101(out_range), no signal (error).

Concerning the Google test, we created a test function that contains three arrays (*input_stimuli_test_1*, *expected_output_test1* and *trigger_test_1*) each of which has been initialized with 2.7 million values obtained by the three objects generated in the Matlab Workspace.

The test is correctly executed as expected and the result that we obtained for the coverage is the follow:

```
File 'feedback_control.c'  
Lines executed:100.00% of 150  
Branches executed:100.00% of 53  
Taken at least once:94.34% of 53  
No calls  
Creating 'feedback_control.c.gcov'
```

As we can see the file '*feedback_control.c*' contains 150 lines. All of these have been taken, as a matter of fact, we reach 100% of statements coverage.

As for decision coverage which is not 100%. This is because during the code generation additional checks, regarding the after conditions, have been added automatically by Simulink.

A naïve solution could be to replace each after condition with a transition with their same temporal behaviour based on a counter variable, but from a desing point of view it is not recommended.

Finally, even if we don't reach a decision coverage of 100% in Google test we can still guarantee that our test has the quality required.

3. Arduino project

The simulink model of the ranging estimator has been deployed to an Arduino UNO using full executable export process.

In particular, we used the blocks *Digital Input* and *Digital Output* provided by the Simulink Support Packpage for Arduino Hardware.

We configured pin inside these blocks according to the physical configuration. We connected PIN number 11 for the echo signal, PIN number 12 for the trigger signal and PIN number 13 for the ArduinoUno LED. In this case, we used a FixedStepDiscete time solver of $100\mu s$ since this is the minimum step size that the hardware could handle, this causes a decrease in the accuracy of the measurements.

The side effect is that we were forced to change the amount of time the trigger signal must be up. We implemented that the trigger will stay in the ON state for $100\mu s$, in this way the signal is no longer 0.06001 but 0.06010. For this reason, we changed the after conditions with the following configuration: *after(0.0601, sec)*.

